
LÓGICA MATEMÁTICA

Cuarto del Grado en Matemáticas

Hugo Marquerie

Profesor: Arturo Rodríguez Fanlo
Facultad de Ciencias - Universidad Autónoma de Madrid
Cuatrimestre

27 de enero de 2026

Puedes encontrar mi colección completa de apuntes en <https://hmarquer.github.io>.

Índice

1. Teoría de modelos

1.1 Lenguajes y estructuras

Definición 1.1.1 (Lenguaje de primer orden). L es un lenguaje de primer orden (FOL) $\iff L$ es un conjunto de símbolos tal que existe una partición de L en:

- Símbolos formales $\{\dot{=}, \top, \neg, \wedge, \exists\}$.
- Símbolos de función: $F = \bigcup_{k=0}^{\infty} F_k$, donde $\forall k \in \mathbb{N} \cup \{0\} : F_k$ es un conjunto de símbolos de función de aridad k .
- Símbolos de relación: $R = \bigcup_{k=0}^{\infty} R_k$, donde $\forall k \in \mathbb{N} \cup \{0\} : R_k$ es un conjunto de símbolos de relación de aridad k .

Definición 1.1.2 (Constante). Sea L un FOL, $c \in L$ es una constante $\iff c$ es un símbolo de función de aridad 0 en L .

Definición 1.1.3 (Proposición). Sea L un FOL, $p \in L$ es una proposición $\iff p$ es un símbolo de relación de aridad 0 en L .

Definición 1.1.4 (Signatura). Sea L un FOL, la signatura de L es $\bigsqcup_k F_k \sqcup \bigsqcup_k R_k \subset L$.

Observación 1.1.5. Un lenguaje de primer orden L queda completamente determinado por su signatura. Por ello, en general, se identifica un lenguaje con su signatura.

Ejemplos 1.1.1 (de lenguajes de primer orden).

- 1 Lenguaje vacío: la signatura es vacía, es decir, solo tenemos los símbolos formales.
- 2 Lenguaje de grupos: viene dado por la signatura $L_{\text{gr}} = \{\cdot, 1, \cdot^{-1}\}$ donde $\cdot$ es un símbolo de función binario, 1 es una constante y $\cdot^{-1}$ es un símbolo de función unario.

Alternativamente, el lenguaje aditivo de grupos es $L_{\text{gr}}^+ = \{+, 0, -\}$.

- 3 El lenguaje de anillos (y cuerpos) es $L_a = \{+, \cdot, -, 0, 1\}$ donde $+$ y $\cdot$ son símbolos de función binarios, $-$ es un símbolo de función unario y 0 y 1 son constantes.
- 4 El lenguaje de órdenes es $L_{<} = \{<\}$ donde $<$ es un símbolo de relación binario.
- 5 El lenguaje de grafos es $L_E = \{E\}$ donde E es un símbolo de relación binario.
- 6 El lenguaje de equivalencia es $L_{\sim} = \{\sim\}$ donde $\sim$ es un símbolo de relación binario.

- [7] Dado un cuerpo K , el lenguaje de K -espacios vectoriales es $L_{K\text{-vec}} = \{+, -, 0\} \cup \{k \cdot\}_{k \in K}$ donde $+$ es un símbolo de función binario, $-$ es un símbolo de función unario, 0 es una constante y $\forall k \in K : k \cdot$ es un símbolo de función unario.
- [8] El lenguaje de la aritmética es $L_{\mathbb{N}} = \{\cdot, +, S, 0, <\}$ donde $\cdot$ y $+$ son símbolos de función binarios, S es un símbolo de función unario, 0 es una constante y $<$ es un símbolo de relación binario.
- El lenguaje de la aritmética con exponencial es la extensión de $L_{\mathbb{N}}$ dada por la signatura $L_{\mathbb{N}, \text{exp}} = \{\cdot, +, \cdot, S, 0, <\}$ donde $\cdot$ es un símbolo de función binario (elevar a la potencia).
- [9] El lenguaje de los reales es el lenguaje extensión del lenguaje de anillos dado por la signatura $L_{\mathbb{R}} = \{+, \cdot, -, 0, 1, <\}$ donde $<$ es un símbolo de relación binario.
- [10] El lenguaje de conjuntos es $L_{\in} = \{\in\}$ donde $\in$ es un símbolo de relación binario.

Es importante remarcar que los elementos de un lenguaje son solo símbolos sin significado alguno (excepto por la distinción formal entre el tipo de símbolo y su aridad). Dicho esto, es evidente que al construir un lenguaje tenemos en mente un uso y, por tanto, elegimos los símbolos de acuerdo con las interpretaciones previstas. Sin embargo, debemos tener presente que estas pretendidas interpretaciones (las cuales definiremos más adelante al hablar de “estructuras”) no están en el lenguaje en sí mismo.

Por ejemplo, conviene notar que, como lenguajes, L_{gr} y L_{agr} son (esencialmente) el mismo lenguaje, en el sentido de que hay una correspondencia biyectiva que preserva los tipos de símbolos y las aridades. En efecto, aunque tengamos en mente utilizar L_{agr} para grupos abelianos y L_{gr} para grupos arbitrarios, los lenguajes en sí mismos no indican esta distinción.

Más aún, los lenguajes $L_{<}$, L_E , $L_{\sim}$ y $L_{\in}$ son esencialmente idénticos. En efecto, aunque implícitamente tenemos en mente utilizar $L_{<}$ para órdenes, L_E para grafos, $L_{\sim}$ para relaciones de equivalencia y $L_{\in}$ para conjuntos, los lenguajes en sí mismos no indican que $<$ sea un orden, E sea una relación de grafo, $\sim$ sea una relación de equivalencia o $\in$ sea la relación de pertenencia. En lo relativo al lenguaje, solo tenemos símbolos formales sin significado.

Definición 1.1.6 (Conjunto de variables). Sea L un lenguaje de primer orden, X es un conjunto de variables para $L \iff X$ es un conjunto disjunto de L .

- A cada elemento de X se le llama variable simple.
- A cada subconjunto de X se le llama variable.
- A cada variable ordenada se le llama tupla de variables sin repetición.

Definición 1.1.7 (Palabra). Sea L un lenguaje de primer orden con variables X , una palabra en L es una tupla finita cuyos elementos son símbolos de $L \cup X$.

Definición 1.1.8 (Término). Sea L un lenguaje de primer orden con variables X , los términos de L son las palabras definidas inductivamente como sigue:

- Caso 0: $\text{Ter}_0(L; X)$ es el conjunto de las constantes y variables simples.
- Caso $n + 1$: $\text{Ter}_{n+1}(L; X)$ es el conjunto que incluye a $\text{Ter}_n(L; X)$ y contiene las palabras de la forma $f t_1 \dots t_k$ donde f es un símbolo de función k -ario y $t_1, \dots, t_k \in \text{Ter}_n(L; X)$.

El conjunto de términos de L se denota por $\text{Ter}(L; X) := \bigcup_{n=0}^{\infty} \text{Ter}_n(L; X)$.

Definición 1.1.9 (Fórmula). Sea L un lenguaje de primer orden con variables X , una fórmula de L es una palabra en $L \cup X$ definida inductivamente como sigue:

- Caso 0: $\text{For}_0(L; X)$ es el conjunto de las palabras de la forma $R t_1 \dots t_m$ donde R es un símbolo de relación de aridad m y $t_1, \dots, t_m \in \text{Ter}(L; X)$. Además, incluye la palabra $\top$ y las palabras de la forma $t_1 t_2$ donde $t_1, t_2 \in \text{Ter}(L; X)$.
- Caso $n + 1$: $\text{For}_{n+1}(L; X)$ es el conjunto que incluye a $\text{For}_n(L; X)$ y contiene las palabras de la forma:
 - (i) $\wedge \varphi_1 \varphi_2$ donde $\varphi_1, \varphi_2 \in \text{For}_n(L; X)$.
 - (ii) $\neg \varphi$ donde $\varphi \in \text{For}_n(L; X)$.
 - (iii) $\exists x \varphi$ donde x es una variable simple y $\varphi \in \text{For}_n(L; X)$.

El conjunto de fórmulas de L se denota por $\text{For}(L; X) := \bigcup_{n \in \mathbb{N}} \text{For}_n(L; X)$.

Ejemplo 1.1.2 (de fórmula). En $L_{\mathbb{R}}$, $\exists x \exists y \wedge \neg \wedge = \cdot x x y < x 0 \top$ es una fórmula. Normalmente, se escribiría como $\exists x, y : (\neg(x^2 = y \wedge x < 0) \wedge \top)$.

Aunque, evidentemente, tenemos un significado en mente a la hora de definir las fórmulas y los términos, por ahora no hemos definido en absoluto qué es eso del “significado” de una fórmula o de un término. En lo que se refiere al lenguaje, las fórmulas y los términos son solo cadenas de símbolos formales construidas siguiendo una serie de reglas sintácticas.

Observación 1.1.10. Si consideramos el caso con $X = \emptyset$ y L un lenguaje proposicional (es decir, que su signature solo contiene símbolos de relación 0-arios), la lógica de primer orden con L se reduce a la lógica proposicional, es decir, $\text{Ter}(L; X) = \emptyset$ y $\text{For}(L; X)$ son las fórmulas en lógica proposicional. De este modo, la lógica de primer orden es una generalización de

la lógica proposicional, también llamada lógica de orden cero.

Notación. Toda biyección entre dos conjuntos de variables induce una biyección entre los respectivos conjuntos de términos y fórmulas. Además, como los términos y las fórmulas utilizan solo un número finito de variables, en general podemos asumir que trabajamos con un número numerable de variables. En consecuencia, a la hora de definir los términos y fórmulas es esencialmente irrelevante la forma concreta del conjunto de variables. Es por ello que, salvo que sea estrictamente necesario, evitaremos especificar explícitamente la familia de variables $\mathbf{X}$ que utilizamos. Por tanto, a partir de ahora, normalmente escribiremos $\text{Ter } L := \text{Ter}(L, \mathbf{X})$ y $\text{For } L := \text{For}(L, \mathbf{X})$.

Por simplicidad, usamos solo los símbolos $\top$ (tautología), $\neg$ (negación), $\wedge$ (conjunción) y $\exists$ (existe) para las definiciones. Dicho esto, en la práctica, también usamos los símbolos $\perp$ (falso), $\vee$ (disyunción), $\rightarrow$ (condicional), $\leftrightarrow$ (equivalencia) y $\forall$ (universal). Estos símbolos los introducimos como abreviaturas del siguiente modo: Escribimos

- $\perp$ para $\neg\top$,
- $\varphi \vee \psi$ para $\neg(\neg\varphi \wedge \neg\psi)$,
- $\varphi \rightarrow \psi$ para $\neg\varphi \vee \psi$,
- $\varphi \leftrightarrow \psi$ para $(\varphi \rightarrow \psi) \wedge (\psi \rightarrow \varphi)$, y
- $\forall x \varphi$ para $\neg \exists x \neg \varphi$.

Por otro lado, en las definiciones formales de términos y fórmulas estamos utilizando la notación polaca (que escribe las funciones, relaciones y conectivas como prefijos). Esta notación tiene la virtud de evitar el uso de paréntesis, pero resulta poco legible para los humanos. Para facilitar la lectura utilizamos la notación estándar: Escribimos

- $f(t_1, \dots, t_m)$ para $f t_1 \dots t_m$ y $(t_1 f t_2)$ para $f t_1 t_2$ si f es binario;
- $R(t_1, \dots, t_m)$ para $R t_1 \dots t_m$ y $(t_1 R t_2)$ para $R t_1 t_2$ si R es binario;
- $(t_1 \doteq t_2)$ para $\doteq t_1 t_2$ y usamos $=$ en lugar de $\doteq$ si no hay ambigüedad;
- $(\varphi \wedge \psi)$ para $\wedge \varphi \psi$, $(\varphi \vee \psi)$ para $\vee \varphi \psi$, $(\varphi \rightarrow \psi)$ para $\rightarrow \varphi \psi$ y $(\varphi \leftrightarrow \psi)$ para $\leftrightarrow \varphi \psi$;
- $(\exists x_1, \dots, x_n \varphi)$ para $\exists x_1 \dots \exists x_n \varphi$ y $(\forall x_1, \dots, x_n \varphi)$ para $\forall x_1 \dots \forall x_n \varphi$; y
- $\bigwedge_{i=1}^n \varphi_i$ para $\wedge \dots \wedge \varphi_1 \dots \varphi_n$ y $\bigvee_{i=1}^n \varphi_i$ para $\vee \dots \vee \varphi_1 \dots \varphi_n$.

La notación estándar es ambigua. Por ejemplo, $p \vee q \wedge r$ puede ser $(p \vee q) \wedge r$ o $p \vee (q \wedge r)$. Para subsanar esto utilizamos los paréntesis. Dicho esto, omitimos los paréntesis siempre que

la omisión no dé lugar a ambigüedades. En este sentido, para reducir al mínimo el uso de paréntesis, seguiremos el convenio de prioridad $\neg \wedge \exists$, según el cual primero actúa $\neg$, luego actúan las conectivas binarias ($\wedge$, $\vee$, $\rightarrow$ o $\leftrightarrow$), y, por último, los cuantificadores ($\exists$ o $\forall$). De este modo, para la fórmula $\exists x ((\neg\varphi) \wedge \psi)$, podemos escribir simplemente $\exists x \neg\varphi \wedge \psi$.

Definición 1.1.11 (Complejidad). Sea L un lenguaje de primer orden, y sea $t \in \text{Ter } L$, $\text{comp}(t)$ es la complejidad de $t \iff \text{comp}(t) = \max\{n \in \mathbb{N} : t \in \text{Ter}_n L\}$.

Sea $\varphi \in \text{For } L$, $\text{comp}(\varphi)$ es la complejidad de $\varphi \iff \text{comp}(\varphi) = \max\{n \in \mathbb{N} : \varphi \in \text{For}_n L\}$

A priori, es posible que una fórmula (o término) se pueda obtener de dos formas distintas a partir de fórmulas (o términos) de menor complejidad. Es decir, en principio, podría darse el caso de que las fórmulas (o los términos) sean ambiguos. Por ejemplo, en notación estándar sin paréntesis, la palabra $\varphi_1 \wedge \varphi_2 \vee \varphi_3$ se puede obtener de dos formas distintas como " $(\varphi_1 \wedge \varphi_2) \vee \varphi_3$ " o como " $\varphi_1 \wedge (\varphi_2 \vee \varphi_3)$ " (es decir, usando primero $\wedge$ y después $\vee$, o usando primero $\vee$ y después $\wedge$).

Del mismo modo, conviene notar que, en principio, la complejidad de los términos y las fórmulas no está unívocamente definida, es decir, en caso de existir ambigüedades, podría darse la situación de que una fórmula (o término) se pueda obtener de dos formas distintas como fórmula (o término) de complejidad distinta. Por ejemplo, con notación estándar sin paréntesis, la fórmula $\varphi_1 \wedge \varphi_2 \wedge \varphi_3 \wedge \varphi_4$ puede corresponderse con la fórmula $(\varphi_1 \wedge \varphi_2) \wedge (\varphi_3 \wedge \varphi_4)$, de complejidad 3, o con la fórmula $\varphi_1 \wedge (\varphi_2 \wedge (\varphi_3 \wedge \varphi_4))$, de complejidad 4.

A continuación vamos a demostrar que este no es el caso en notación polaca, es decir, que no existen ambigüedades en las fórmulas y los términos tal y como los hemos definido usando notación polaca.

Lema 1.1.1. *Sea L un lenguaje de primer orden, $\eta_1, \dots, \eta_n$ y $\eta'_1, \dots, \eta'_m$ todos términos o todas fórmulas. Entonces, las palabras $\eta_1 \dots \eta_n$ y $\eta'_1 \dots \eta'_m$ son iguales si y solo si $n = m$ y $\eta_1 = \eta'_1, \dots, \eta_n = \eta'_n$.*

Demostración: La implicación de derecha a izquierda es evidente. Demostramos la implicación de izquierda a derecha.

Tenemos que $\eta_1 \dots \eta_n = \eta'_1 \dots \eta'_m$. Sin pérdida de generalidad, asumimos que $m \geq n$. Evidentemente, es suficiente demostrar que $\eta_1 = \eta'_1, \dots, \eta_n = \eta'_n$, pues en tal caso $m = n$. Supongamos por contradicción que $\eta_i \neq \eta'_i$ para algún $i \in \{1, \dots, n\}$ y tomamos $k = \min\{i \in \mathbb{N}_n : \eta_i \neq \eta'_i\}$. Entonces, en particular, $\eta_1 \dots \eta_{k-1} = \eta'_1 \dots \eta'_{k-1}$, luego eliminando los símbolos iniciales, tenemos que $\eta_k \dots \eta_n = \eta'_k \dots \eta'_m$.

Si $|\eta_k| > |\eta'_k|$ (donde $|\cdot|$ indica la longitud de la palabra), eliminando los símbolos finales, tenemos que $\eta_k = \eta'_k \zeta$ con ζ una palabra no vacía. Simétricamente, si $|\eta'_k| > |\eta_k|$, tenemos que $\eta'_k = \eta_k \zeta$ para una palabra ζ no vacía. En consecuencia, es suficiente demostrar lo siguiente:

Afirmación: Ningún fragmento inicial propio de un término es un término. Ningún fragmento inicial propio de una fórmula es una fórmula.

- (a) Términos: Sea η un término y $\eta = \eta' \zeta$ con η' un término. Demostramos por inducción en la longitud de η que $\eta = \eta'$ y $\zeta = \emptyset$. Si $|\eta| = 1$, como η' es un término, tenemos que $1 \leq |\eta'| \leq |\eta| = 1$, luego $|\eta'| = |\eta| = 1$ y $|\zeta| = 0$.

Supongamos que $|\eta| = l > 1$ y se cumple para todo término de longitud menor a l . Como $|\eta| > 1$, tenemos que $\eta = ft_1 \dots t_k$ para cierto símbolo de función f de aridad k ($k \neq 0$) y ciertos términos $t_1, \dots, t_k$. Como $\eta = \eta' \zeta$, concluimos que el primer símbolo de η' es f . Como η' es un término, tenemos que $\eta' = ft'_1 \dots t'_k$ para ciertos términos $t'_1, \dots, t'_k$. Eliminando el primer símbolo, obtenemos que $t_1 \dots t_k = t'_1 \dots t'_k \zeta$.

Supongamos que existe i tal que $t_i \neq t'_i$. Tomamos $m := \min\{i : t_i \neq t'_i\}$. En tal caso, como $t_1 \dots t_{m-1} = t'_1 \dots t'_{m-1}$, eliminando los símbolos iniciales, tenemos que $t_m \dots t_k = t'_m \dots t'_k \zeta$. Si $|t_m| > |t'_m|$, entonces, eliminando los últimos símbolos, tenemos $t_m = t'_m \zeta'$ con cierta palabra ζ' . Ahora bien, $|t_m| < l$, luego $\zeta' = \emptyset$ y $t_m = t'_m$ por hipótesis de inducción. Si por el contrario $|t_m| < |t'_m|$, entonces, eliminando los últimos símbolos, tenemos $t'_m = t_m \zeta'$ con cierta palabra ζ' . Como $|t'_m| < l$, concluimos igualmente que $\zeta' = \emptyset$ y $t_m = t'_m$. En ambos casos, hemos llegado a una contradicción con $t_m \neq t'_m$. Por tanto, $t_1 = t'_1, \dots, t_k = t'_k$.

Entonces, como $\eta = ft_1 \dots t_k = \eta' \zeta = \emptyset$.

- (b) Fórmulas: Sea η una fórmula y $\eta = \eta' \zeta$ con η' una fórmula. Demostramos por inducción en la longitud de η que $\eta = \eta'$ y $\zeta = \emptyset$. Si $|\eta| = 1$, como η' es una fórmula, tenemos que $0 < |\eta'| \leq |\eta| = 1$, luego $|\eta'| = |\eta| = 1$ y $|\zeta| = 0$.

Supongamos que $|\eta| = l > 1$ y se cumple para toda fórmula de longitud menor a l .

Consideremos primero el caso de fórmulas de complejidad 0. Digamos que $\eta = Rt_1 \dots t_m$ con R símbolo de relación m -ario ($\doteq$ y $m = 2$). Como $\eta = \eta' \zeta$, concluimos que el primer símbolo de η' es R . Como η' es una fórmula, tenemos que $\eta' = Rt'_1 \dots t'_m$. Entonces, $t_1 \dots t_m = t'_1 \dots t'_m \zeta$. Ahora bien, por el caso de términos ya demostrado, obtenemos que $t_1 = t'_1, \dots, t_m = t'_m$ y $\zeta = \emptyset$. Por tanto, $\eta = \eta'$.

Consideremos $\eta = \neg\varphi$. Como $\eta = \eta'\zeta$, concluimos que el primer símbolo de η' es $\neg$. Por tanto, como η' es una fórmula, $\eta' = \neg\phi$ con ϕ fórmula. Entonces, $\varphi = \phi\zeta$ con $|\varphi| < l$. Entonces, por hipótesis de inducción, $\varphi = \phi$ y $\zeta = \emptyset$, luego $\eta = \eta'$.

Consideremos $\eta = \exists x\varphi$. Como $\eta = \eta'\zeta$, concluimos que los dos primeros símbolos de η' son $\exists x$. Por tanto, como η' es una fórmula, $\eta' = \exists x\phi$ con ϕ fórmula. Entonces, $\varphi = \phi\zeta$ con $|\varphi| < l$. Entonces, por hipótesis de inducción, $\varphi = \phi$ y $\zeta = \emptyset$, luego $\eta = \eta'$.

Finalmente, consideremos $\eta = \varphi_1 \wedge \varphi_2$. Como $\eta = \eta'\zeta$, concluimos que el primer símbolo de η' es $\wedge$. Por tanto, como η' es una fórmula, $\eta' = \wedge\phi_1\phi_2$. O bien $|\varphi_1| \geq |\phi_1|$ o $|\varphi_1| \leq |\phi_1|$. Eliminando los símbolos finales, tenemos que $\varphi_1 = \phi_1\zeta'$ o $\phi_1 = \varphi_1\zeta'$ para alguna palabra ζ' . Como $|\varphi_1| < l$ y $|\phi_1| < l$, concluimos por hipótesis de inducción que $\varphi_1 = \phi_1$. Ahora, como $\varphi_2 = \phi_1\phi_2\zeta$, eliminando los símbolos iniciales, tenemos que $\varphi_2 = \phi_2\zeta$. Como $|\varphi_2| < l$, concluimos que $\varphi_2 = \phi_2$ y $\zeta = \emptyset$ por hipótesis de inducción. En consecuencia, $\eta = \eta'$ y $\zeta = \emptyset$. ■

Temporary page!

L^AT_EX was unable to guess the total number of pages correctly. As there was some unprocessed data that should have been added to the final page this extra page has been added to receive it.

If you rerun the document (without altering it) this surplus page will go away, because L^AT_EX now knows how many pages to expect for this document.